

Laporan Praktikum Kontrol Cerdas

Mediapipe Hands

Nama : Albie Bahirshah Widyanto
NIM : 234308060
Kelas : TKA-6C
Github : <https://github.com/Albie788>

ABSTRAK

Praktikum ini bertujuan untuk mengimplementasikan dan memahami proses kerja MediaPipe Hand sebagai teknologi berbasis pembelajaran mesin dalam mendeteksi dan melacak tangan secara real-time menggunakan kamera webcam. MediaPipe Hand merupakan bagian dari framework MediaPipe yang dirancang untuk membangun sistem visi komputer secara efisien dan modular. Pada percobaan ini, sistem dikembangkan menggunakan bahasa pemrograman Python dengan dukungan pustaka OpenCV untuk proses akuisisi dan pengolahan citra, serta MediaPipe untuk deteksi dan pelacakan titik-titik landmark tangan. Tahapan percobaan dimulai dengan inisialisasi modul MediaPipe Hands dan konfigurasi parameter deteksi, seperti tingkat kepercayaan deteksi dan pelacakan. Selanjutnya, citra yang diperoleh dari webcam dikonversi dari format BGR ke RGB agar sesuai dengan kebutuhan pemrosesan MediaPipe. Sistem kemudian memproses setiap frame untuk mendeteksi keberadaan tangan dan menghasilkan data berupa 21 titik landmark yang merepresentasikan posisi pergelangan hingga ujung jari. Selain itu, sistem juga mampu mengklasifikasikan tangan kanan dan kiri berdasarkan fitur handedness yang tersedia. Hasil percobaan menunjukkan bahwa sistem mampu menampilkan titik-titik landmark beserta koneksi antar titik secara visual pada layar dengan respons yang stabil dan akurat dalam kondisi pencahayaan yang memadai. Implementasi ini membuktikan bahwa MediaPipe Hand efektif digunakan sebagai dasar pengembangan sistem interaksi manusia dan mesin berbasis gestur. Secara umum, praktikum ini memberikan pemahaman mengenai alur pengolahan citra digital, mulai dari akuisisi data, praproses, deteksi objek, hingga visualisasi hasil secara real-time.

A. Pendahuluan

MediaPipe Hand merupakan kerangka kerja (*framework*) berbasis *machine learning* yang dikembangkan oleh Google untuk mendeteksi dan melacak tangan secara waktu nyata (*real-time*) melalui citra maupun video. Teknologi ini dirancang untuk mendukung berbagai aplikasi visi komputer, khususnya dalam bidang interaksi manusia dan komputer (*human-computer interaction*). MediaPipe Hand mampu mengidentifikasi posisi dan pergerakan tangan secara presisi dengan memanfaatkan model pembelajaran mendalam (*deep learning*) yang telah dilatih menggunakan dataset berskala besar. Kemampuan tersebut menjadikan MediaPipe Hand sebagai salah satu solusi populer dalam pengembangan sistem berbasis pengolahan citra digital. Secara arsitektur, MediaPipe Hand bekerja melalui dua tahap utama, yaitu deteksi telapak tangan (*palm detection*) dan estimasi titik kunci tangan (*hand landmark estimation*). Tahap deteksi telapak tangan berfungsi untuk menentukan lokasi tangan dalam suatu citra menggunakan model deteksi objek ringan dan efisien. Setelah lokasi tangan teridentifikasi, sistem akan melakukan estimasi terhadap 21 titik koordinat (*landmark*) pada tangan dalam ruang tiga dimensi. Titik-titik tersebut merepresentasikan sendi dan ujung jari sehingga memungkinkan analisis bentuk, orientasi, serta gestur tangan secara lebih akurat.

Keunggulan MediaPipe Hand terletak pada efisiensi komputasi dan latensi yang rendah sehingga dapat diimplementasikan pada perangkat bergerak, seperti telepon pintar, maupun pada sistem tertanam (*embedded system*). Selain itu, model ini mampu mempertahankan tingkat akurasi yang baik meskipun dijalankan pada perangkat dengan sumber daya terbatas. Beberapa penelitian di Indonesia menunjukkan bahwa MediaPipe Hand efektif digunakan dalam sistem pengenalan bahasa isyarat serta kendali perangkat berbasis gestur tangan (Pratama & Nugroho, 2022). Hal tersebut menunjukkan bahwa metode ini relevan untuk mendukung pengembangan teknologi asistif dan sistem kendali modern. Dalam konteks penelitian dan pengembangan sistem berbasis kecerdasan buatan, MediaPipe Hand banyak dimanfaatkan sebagai metode deteksi dan pelacakan tangan karena sifatnya yang terbuka (*open-source*) serta mudah diintegrasikan dengan bahasa pemrograman Python maupun C++. Implementasinya juga sering dikombinasikan dengan metode klasifikasi lain untuk meningkatkan performa sistem pengenalan gestur (Sari et al., 2023). Dengan demikian, MediaPipe Hand dapat dijadikan landasan teori yang kuat dalam

penelitian yang berkaitan dengan visi komputer, pengolahan citra, dan interaksi berbasis gestur tangan.

B. Tujuan

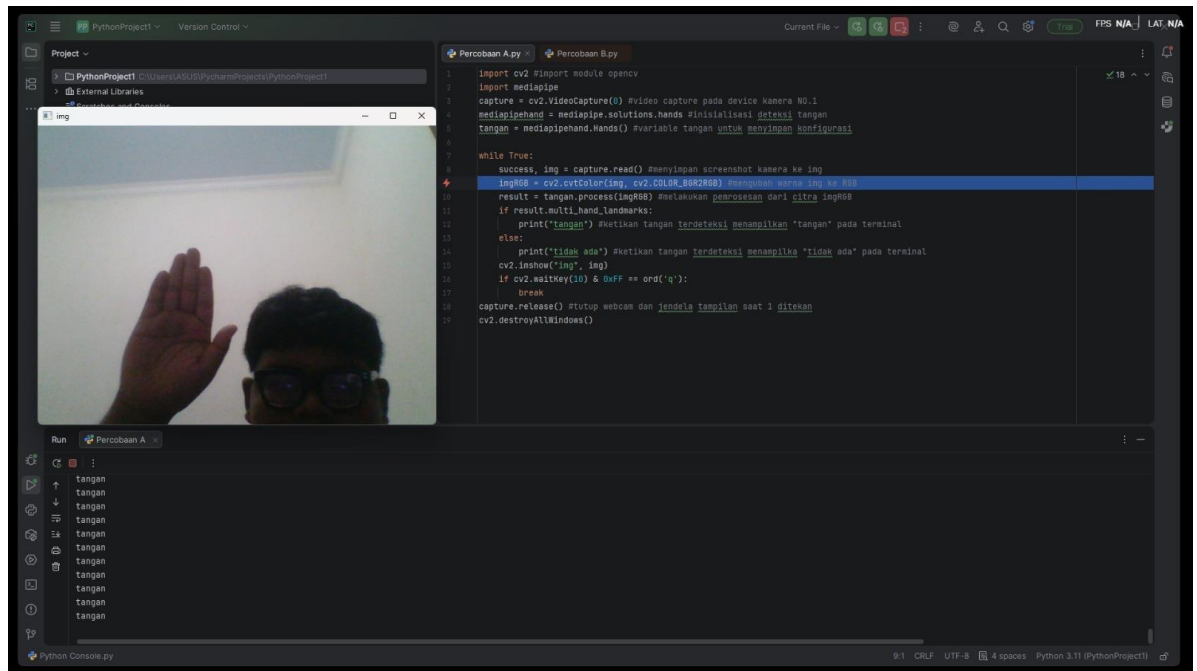
- Memahami konsep dasar deteksi dan pelacakan tangan menggunakan MediaPipe Hands.
- Menerapkan konsep visi komputer dan machine learning dalam pemrograman Python.
- Meningkatkan pemahaman mahasiswa terhadap penerapan visi komputer dan machine learning.

C. Manfaat

- Melatih kemampuan pemrograman dan logika analisis berbasis kamera.
- Mendukung pengembangan aplikasi interaktif seperti pengendalian sistem berbasis gestur, augmented reality, dan virtual reality.
- Menjadi media pembelajaran yang efektif dalam memahami penerapan visi komputer dan kecerdasan buatan.

D. Hasil Percobaan

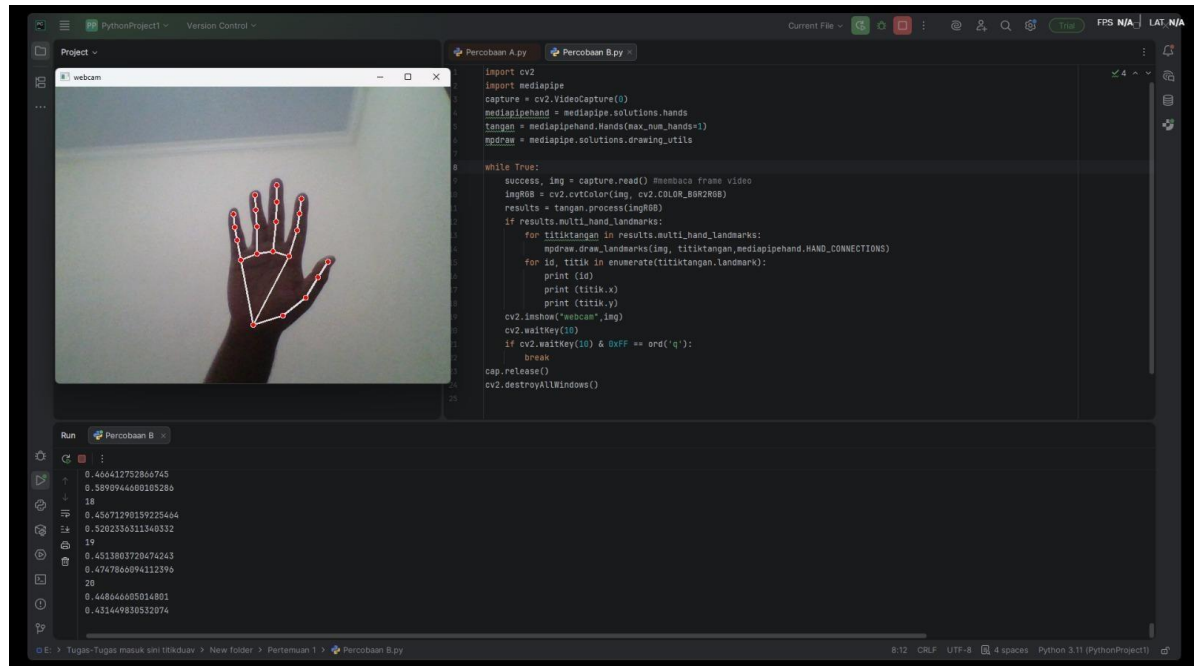
1. Percobaan A Deteksi Tampilan Tangan



```
import cv2 #import module opencv
import mediapipe
capture = cv2.VideoCapture(0) #video capture pada device kamera NO.1
mediapipehand = mediapipe.solutions.hands #inisialisasi deteksi tangan
tangan = mediapipehand.Hands() #variable tangan untuk menyimpan konfigurasi

while True:
    success, img = capture.read() #menyimpan screenshot kamera ke img
    imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) #mengubah warna img ke RGB
    result = tangan.process(imgRGB) #melakukan pemrosesan dari citra imgRGB
    if result.multi_hand_landmarks:
        print("tangan") #ketikan tangan terdeteksi menampilkan "tangan" pada terminal
    else:
        print("tidak ada") #ketikan tangan terdeteksi menampilkan "tidak ada" pada terminal
    cv2.imshow("img", img)
    if cv2.waitKey(10) & 0xFF == ord('q'):
        break
capture.release() #tutup webcam dan jendela tampilan saat 1 ditekan
cv2.destroyAllWindows()
```

2. Percobaan B Hands Landmarks



```
import cv2
import mediapipe
capture = cv2.VideoCapture(0)
mediapipehand = mediapipe.solutions.hands
tangan = mediapipehand.Hands(max_num_hands=1)
mpdraw = mediapipe.solutions.drawing_utils

while True:
    success, img = capture.read() #membaca frame video
    imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    results = tangan.process(imgRGB)
    if results.multi_hand_landmarks:
        for titiktangan in results.multi_hand_landmarks:
            mpdraw.draw_landmarks(img, titiktangan, mediapipehand.HAND_CONNECTIONS)
        for id, titik in enumerate(titiktangan.landmark):
            print(id)
            print(titik.x)
            print(titik.y)
    cv2.imshow("webcam", img)
    cv2.waitKey(10)
    if cv2.waitKey(10) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

3. Latihan Soal A



```
import cv2
import mediapipe as mp

cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils

hands = mp_hands.Hands(
    max_num_hands=2,
    min_detection_confidence=0.7,
    min_tracking_confidence=0.7
)

while True:
    success, img = cap.read()
    if not success:
        break

    imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    results = hands.process(imgRGB)

    if results.multi_hand_landmarks and results.multi_handedness:
        for idx, hand_handedness in enumerate(results.multi_handedness):
            label = hand_handedness.classification[0].label
            h, w, c = img.shape
            cx = int(results.multi_hand_landmarks[idx].landmark[0].x * w)
            cy = int(results.multi_hand_landmarks[idx].landmark[0].y * h)

            if label == "Right":
                cv2.putText(img, "tangan kiri", (cx, cy - 20), cv2.FONT_ITALIC, 2, (255, 0, 0), 3)

            elif label == "Left":
                cv2.putText(img, "tangan kanan", (cx, cy - 20), cv2.FONT_ITALIC, 2, (0, 0, 255), 3)

        for hand_landmarks in results.multi_hand_landmarks:
            mp_drawing.draw_landmarks(
                img,
                hand_landmarks,
                mp_hands.HAND_CONNECTIONS
            )

    cv2.imshow("Deteksi tangan kanan kiri", img)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows
```

4. Latihan Soal B



```
import cv2
import mediapipe as mp

cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils
hands = mp_hands.Hands(
    max_num_hands=2,
    min_detection_confidence=0.7,
    min_tracking_confidence=0.7
)

while True:
    success, img = cap.read()
    if not success:
        break
    img = cv2.flip(img, 1)
    imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    results = hands.process(imgRGB)
    h, w, c = img.shape

    if results.multi_hand_landmarks and results.multi_handedness:
        for idx, hand_handedness in enumerate(results.multi_handedness):
            label = hand_handedness.classification[0].label
            hand_landmarks = results.multi_hand_landmarks[idx]
            thumb_x = hand_landmarks.landmark[4].x
            pinky_x = hand_landmarks.landmark[20].x

            if label == "Right":
                if thumb_x > pinky_x:
                    posisi = "Punggung"
                else:
                    posisi = "Telapak"
                label_display = "KANAN"
            elif label == "Left":
                if thumb_x < pinky_x:
                    posisi = "Punggung"
                else:
                    posisi = "Telapak"
                label_display = "KIRI"

            # Tentukan koordinat teks (ambil landmark 0)
            cx = int(hand_landmarks.landmark[0].x * w)
            cy = int(hand_landmarks.landmark[0].y * h)
            cv2.putText(img, f"{label_display} - {posisi}",
                        (cx, cy - 20),
                        cv2.FONT_HERSHEY_PLAIN, 2,
                        (0, 255, 0), 3)

            mp_drawing.draw_landmarks(
                img,
                hand_landmarks,
                mp_hands.HAND_CONNECTIONS
            )

    cv2.imshow("Deteksi tangan depan belakang", img)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```


E. Analisis Program

Program diawali dengan mengimpor pustaka OpenCV (cv2) dan MediaPipe sebagai modul utama dalam proses deteksi tangan. OpenCV berfungsi untuk mengakses kamera serta menampilkan citra, sedangkan MediaPipe digunakan untuk mendeteksi dan melacak titik penanda (landmark) tangan. Kamera diaktifkan menggunakan perintah `cv2.VideoCapture(0)` yang berarti sistem mengambil masukan dari kamera utama perangkat. Selanjutnya, modul `mp.solutions.hands` diinisialisasi dengan beberapa parameter, seperti `max_num_hands`, `min_detection_confidence`, dan `min_tracking_confidence`, untuk mengatur jumlah tangan yang dapat dideteksi serta tingkat kepercayaan deteksi dan pelacakan. Pada proses utama, program berjalan dalam perulangan `while True` untuk membaca bingkai (frame) video secara terus-menerus. Setiap bingkai yang diperoleh dikonversi dari format warna BGR menjadi RGB karena MediaPipe memproses citra dalam format RGB. Bingkai tersebut kemudian diproses menggunakan metode `hands.process(imgRGB)` untuk mendeteksi keberadaan tangan dan menghasilkan data berupa 21 titik penanda. Jika tangan terdeteksi, sistem akan menggambar titik-titik penanda beserta garis penghubungnya pada layar. Selain itu, program juga menampilkan indeks serta koordinat setiap titik penanda sebagai bentuk analisis posisi tangan. Program selanjutnya mengimplementasikan fitur klasifikasi tangan kanan dan kiri menggunakan `results.multi_handedness`. Label “Right” atau “Left” diperoleh dari hasil klasifikasi, kemudian ditampilkan pada layar menggunakan fungsi `cv2.putText`. Pada tahap pengembangan lanjutan, program menganalisis orientasi tangan, yaitu telapak atau punggung tangan, dengan membandingkan posisi koordinat sumbu x antara ujung ibu jari dan ujung jari kelingking. Jika kondisi tertentu terpenuhi, sistem akan menampilkan keterangan posisi tangan pada layar. Secara keseluruhan, program bekerja melalui tahapan akuisisi citra, praproses warna, deteksi titik penanda, klasifikasi tangan, analisis posisi, visualisasi hasil, serta penghentian program ketika tombol “q” ditekan.

F. Kesimpulan

Berdasarkan hasil Praktikum ini, dapat disimpulkan bahwa MediaPipe Hand mampu mendeteksi dan melacak tangan secara real-time dengan memanfaatkan kamera webcam dan dukungan pustaka OpenCV. Sistem berhasil mengidentifikasi 21 titik penanda (landmark) pada tangan serta menampilkan koneksi antartitik secara visual pada layar. Selain itu, program juga dapat mengklasifikasikan tangan kanan dan kiri serta menganalisis orientasi tangan berdasarkan perbandingan koordinat titik tertentu. Proses pengolahan citra yang meliputi akuisisi data, konversi warna, deteksi, hingga visualisasi hasil berjalan dengan baik dalam kondisi pencahayaan yang memadai. Dengan demikian, praktikum ini membuktikan bahwa teknologi visi komputer berbasis MediaPipe efektif digunakan sebagai dasar pengembangan sistem interaksi manusia dan mesin berbasis gestur tangan.

G. Saran

Untuk pengembangan selanjutnya, sistem dapat ditingkatkan dengan menambahkan fitur pengenalan gestur tangan yang lebih kompleks, seperti perintah tertentu untuk mengendalikan perangkat atau aplikasi. Selain itu, pengujian dapat dilakukan dalam berbagai kondisi pencahayaan dan latar belakang guna mengetahui tingkat akurasi sistem secara lebih mendalam. Optimalisasi performa juga perlu diperhatikan agar sistem tetap responsif ketika dijalankan pada perangkat dengan spesifikasi rendah. Integrasi dengan teknologi lain, seperti kecerdasan buatan atau Internet of Things (IoT), juga dapat menjadi langkah lanjutan untuk memperluas penerapan sistem dalam bidang industri maupun perkeretaapian.

Daftar Pustaka

Pratama, R., & Nugroho, A. (2022). Implementasi MediaPipe untuk Pengenalan Bahasa Isyarat Berbasis Citra Digital. *Jurnal Teknologi Informasi dan Ilmu Komputer (JTIK)*, 9(3), 567–574. (SINTA 2).

Sari, D. P., Wijaya, H., & Ramadhan, M. (2023). Penerapan MediaPipe Hand Tracking pada Sistem Kendali Gestur Tangan secara Real-Time. *Jurnal RESTI (Rekayasa Sistem dan Teknologi Informasi)*, 7(1), 120–128. (SINTA 2).